

CANChat ? Open WebUI Gap Analysis

How Far Behind

- **Commit gap:** 'origin/main' has ~1,177 commits not in 'upstream/main'; upstream has ~6,237 not in CANChat. Divergence is substantial.
- **Diff surface:** Large churn across backend core (config/env, routers, models/migrations, retrieval/vector stack, utils), frontend Svelte UI ('src' apps, admin/chat/layout/workspace), and translations. Infra deltas are smaller (Docker/compose).
- **Merge test:** A trial merge surfaced conflicts in most backend files, many Svelte components, i18n locales, lockfiles, and Dockerfile; CI workflows and LICENSE/README also conflicted.

CANChat Customizations (patterns from CANChat-only commits)

- **Core product / business logic (majority):**
 - Retrieval improvements (async/hybrid search, weighted interleaving, concurrency settings).
 - Metrics recording refinements (non-streaming responses, missing metric handling).
 - Chat lifecycle tweaks (cleanup, preservation for pinned/archived, logging).
 - Reasoning filter for title/tag generation.
 - UX logic for metrics date validation and cleanup settings.
- **Config / infra:**
 - Scheduler version/logging tweaks; occasional release bumps; minimal docker/compose drift.
- **Branding / copy / docs:**
 - CHANGELOG/doc updates; translation wording tweaks (EN/FR and others).
- **Auth / security:** No standout CANChat-only auth/permission commits in the sampled set.

Recommended Migration Approach

1. **Prep:** Branch from 'main'; re-fetch 'upstream/main'; align toolchains (Node/Python) with upstream; capture current env/secrets.
2. **Backend-first merge:** Reconcile in slices? config/env/constants ? routers/handlers ? models/migrations ? retrieval/vector ? utils/storage. Apply upstream migrations in a test DB, then reapply CANChat-specific migrations; plan data migration where schemas diverge.
3. **Frontend merge:** Align 'src/lib/apis' with upstream API shape; then merge UI areas in batches (chat/channel, admin, workspace, layout/shared components). Reconcile i18n by starting from upstream locales and reapplying CANChat copy where needed.
4. **Infra/dep alignment:** Merge 'Dockerfile'/compose and CI workflows; reintroduce CANChat env/logging specifics; regenerate lockfiles ('package-lock', 'pyproject/uv', 'backend/requirements*').
5. **Branding/copy pass:** After code alignment, reapply CANChat branding, README/docs updates, and translation tweaks.
6. **Validation:** Backend tests/migrations, frontend build/tests, E2E smoke (chat, retrieval, admin), staging with prod-like data. Fix regressions, repeat as needed.
7. **Cutover:** Freeze changes, tag/release notes, deploy with rollback plan; monitor.

Effort Estimate (dev-weeks)

- **Lower:** 3?4 wks ? assumes clean reapplication of CANChat logic, minimal schema clashes, fast conflict resolution, quick validation.
- **Realistic:** 6?8 wks ? substantial manual merges in backend retrieval/routers/models and frontend Svelte, multiple test/fix cycles, some data migration work.
- **Upper:** 10?12+ wks ? if schema/data drift is large, E2E regressions pile up, or compliance/review loops add rework; heavy branding/i18n reconciliation.

Key Risks